

VidChopper CLI Mode Architecture and Implementation Plan

Qt-free command-line mode for deterministic video chopping, batch automation, and AI-agent harnesses

Manager decision frame

Build CLI mode as a long-term Qt-free product boundary, not as a hidden shortcut into the GUI. The CLI must be deterministic, scriptable, packageable with the Windows release, and safe for power users, AI agents, and future automation harnesses.

Area	Decision
Architecture path	Choose Option B: no GUI means no Qt. CLI services must be reusable without the Qt SDK or Qt runtime.
Executable name	Ship VidChopperCLI.exe beside VidChopper.exe in the Windows release zip.
Primary workload	Chop one or many videos using one or many explicit JSON/YAML chapter configuration files.
Automation goal	Support power-user scripts, AI coding agents, test harnesses, and future app/plugin embedding.
Default command posture	Invoke from the command line and execute immediately; also support an already-running shell mode with chop commands.

Prepared for Devin Thomas as a manager-facing implementation guide for competent C++ engineers and AI coding agents working on devin-thomas/vid-chopper.

1. Executive Decision

The CLI mode should be a first-class VidChopper entry point with a non-Qt implementation path. The existing GUI remains the human editing surface. The new CLI becomes the deterministic automation surface.

Decision point	Standard
Qt dependency	The CLI target must not link Qt or reuse Qt-only service classes. Shared logic must live in Qt-free code.
Config requirement	A chapter source is explicit. A normal chop command requires JSON/YAML config, manual chapters, CSV, or an explicit embedded-chapter mode.
Batch scope	Support 1:1, N:1, and N:N. Reject 1:N because one source video should not fan out across multiple conflicting configs.
Directory matching	Directory-to-directory is N:N only, matched by exact filename stem. Duplicate stems on either side are an error.
Overwrite behavior	Overwrite by default and emit human-readable messages when an existing output file is replaced.
Progress	Human-readable chapter progress and summary output. No JSON-lines progress in the first release.
Dry run	Required. Print planned clips, output files, and ffmpeg commands without exporting.

Product thesis

CLI mode should make VidChopper easier to automate, not harder to reason about. Every accepted invocation should resolve to a clear list of source/config jobs before any ffmpeg process starts.

2. Current Repo Foundation

The current repository is already positioned for this feature because the non-visual domain model is separated from the Qt Widgets shell. The plan should preserve that separation and extend it into a command-line architecture.

Observed repo area	CLI relevance
src/core	Already contains Qt-free C++20 domain logic for chapter planning, validation, timestamp handling, file naming, and command construction. This remains the base of CLI mode.
src/qt/services	Contains useful behavior, but it is Qt-bound through QProcess, QString, signals, and Qt JSON classes. Reuse behavior by refactoring, not by linking Qt.
ExportSettings	Already models the settings CLI needs: ffmpeg/ffprobe paths, naming patterns, encoder options, overwrite

Observed repo area	CLI relevance
	mode, seek mode, manifests, verification, and default chapter behavior.
Command builder	Already produces deterministic ffmpeg token vectors. CLI dry-run and export should use this same command construction path.
Packaging workflow	The Windows release zip already packages the GUI executable and runtime files. Add the CLI binary to the same release bundle.

Implementation note: do not move Qt code into core. Instead, create new Qt-free services that both the CLI and, over time, the GUI can call through thin adapters.

3. Product Goals and Non-Goals

Goal	Meaning
Power-user first	Commands should be predictable, composable in scripts, and useful without opening the GUI.
Agent/harness friendly	The CLI should accept structured config files, have deterministic validation errors, support dry runs, and return meaningful exit codes.
Config-driven	JSON and YAML are the primary interface because they are easy for humans and AI agents to generate and review.
Batch capable	The CLI must handle a directory of videos against one config or a validated directory of matching configs.
GUI-aligned output	Naming, output directory behavior, ffmpeg command semantics, and manifests should align with the GUI unless explicitly overridden.

Non-goal	Boundary
No Qt runtime dependency	Do not depend on QApplication, QProcess, QString, QSettings, or Qt JSON in the CLI target.
No interactive editing table	CLI is not a terminal clone of the GUI chapter grid. Structured files are the editing interface.
No 1:N video-to-config fanout	A single video plus a config directory is ambiguous and should fail with a clear message.
No broad advanced settings surface in v1	For now expose only CRF, CQ, preset, and threads. Keep encoder/audio/container/seek defaults fixed unless the plan is explicitly revised.
No arbitrary extra ffmpeg args in v1	Extra args increase support complexity and can break command assumptions. Add later only with command-builder tests.

4. Command Model

The CLI should support two entry styles: direct execution for scripts and a lightweight shell mode for already-running sessions. Direct execution is the primary contract.

Mode	Example	Behavior
Direct 1:1	VidChopperCLI.exe input.mkv chapters.yaml	Probe input.mkv, parse chapters.yaml, validate, export immediately.
Direct N:1	VidChopperCLI.exe videos\shared_layout.json	Apply one chapter layout to every supported video in the directory.
Direct N:N	VidChopperCLI.exe videos\chapter_configs\	Pair videos and configs by exact filename stem after validating counts and duplicate stems.
Direct embedded	VidChopperCLI.exe input.mkv --embedded	Use embedded chapters explicitly. If none exist, fail with a clear message.
Directory embedded	VidChopperCLI.exe videos\ --embedded	Probe every supported video and chop only those with embedded chapters, reporting skipped files.
Shell mode	VidChopperCLI.exe --shell; chop input.mp4 chapters.json	Start a persistent command loop. The shell command grammar mirrors direct mode after chop.

```
# Preferred direct usage
VidChopperCLI.exe input.mkv chapters.yaml
VidChopperCLI.exe D:\clips\source_videos D:\clips\chapter_configs --dry-run
VidChopperCLI.exe D:\clips\source_videos shared_chapters.json --threads 4 --crf 20
VidChopperCLI.exe input.mp4 --embedded

# Shell mode
VidChopperCLI.exe --shell
vidchopper> chop input.mp4 chapters.json
vidchopper> chop D:\videos D:\configs --dry-run
vidchopper> exit
```

No-config hint behavior

If a user runs a normal chop command without a chapter source and the source has embedded chapters, do not silently use them. Fail and print the exact command to rerun, for example:
VidChopperCLI.exe input.mkv --embedded.

5. Workload Matrix and Matching Rules

Video argument	Config argument	Result
video file	config file	Allowed: 1:1. The config applies to that video.
video directory	config file	Allowed: N:1. The same layout applies to each supported video. Each job is still validated against its own duration.
video directory	config directory	Allowed: N:N only if every video stem has exactly one config stem and every config stem has exactly one video stem.
video file	config directory	Rejected: 1:N is not supported. User must choose a single config.
video file	--embedded	Allowed if embedded chapters exist. Otherwise fail.
video directory	--embedded	Allowed. Process videos with embedded chapters and skip/report videos without them.

5.1 Directory Matching Standard

- Supported video extensions in the first implementation should include .mp4, .mkv, and .mov to match the current source-container behavior. The examples can emphasize .mp4 and .mkv because those are the expected power-user directories.
- Supported config extensions are .json, .yaml, and .yml.
- Duplicate video stems are fatal: show foo.mp4 and foo.mkv as the conflict rather than picking one.
- Duplicate config stems are fatal: show foo.json and foo.yaml as the conflict rather than picking one.
- Directory-to-directory count mismatch is fatal before export. The error should list missing configs and orphan configs.
- File matching is case-insensitive on Windows but error messages should preserve the original filenames.

6. Configuration File Contract

JSON and YAML should share one schema. YAML is a syntax choice, not a separate product model. Parsed config should normalize into a project-owned CliChapterConfig or equivalent C++ aggregate before validation.

```
version: 1
chapters:
- name: Intro
```

```

start: "00:00:00.000"
end: "00:02:30.000"
- name: Match 1
start: "00:02:30.000"
end: "00:18:45.500"
output:
directory: "%source%_chapters"
namingPattern: "%index% - %name%"
settings:
crf: 18
cq: 22
x264Preset: slow
nvencPreset: p5
threads: 0
manifests:
json: true
csv: false

```

Field	Required	Standard
version	Yes	Integer schema version. Start with 1 so future schema changes can be explicit.
chapters	Yes for config files	Ordered chapter list. Each chapter must include name, start, and end.
chapters[].name	Yes	Trimmed non-empty display/file-name source. Existing filename sanitization still applies.
chapters[].start/end	Yes	Accept HH:MM:SS.mmm strings and numeric milliseconds. Reject ambiguous free-form times.
output.directory	No	Defaults to the GUI-aligned output folder pattern. Allow literal paths or pattern tokens.
output.namingPattern	No	Defaults to %index% - %name%, matching the GUI.
settings.crf/cq/presets/threads	No	Only first-wave encoder tunables. Everything else stays on default behavior.
manifests.json/csv	No	JSON default true. CSV default false. CLI flags may override.

6.1 Config Ownership Rules

- The command-line video path is authoritative. If a future config file contains a source path, it should be optional metadata and must match the command-line source when both are present.

- N:1 configs must not hard-code source-specific duration assumptions unless validation passes for every video.
- The parser should report all schema errors found in a file when practical, not just the first missing key.
- CSV and manual chapters can be added as convenience inputs after the JSON/YAML contract is stable. They should normalize into the same config model.

7. Defaults, Flags, and Output Behavior

Area	Initial CLI standard
Encoder	Auto. Use the same resolve_encoder logic and prefer HEVC NVENC only when auto-detection confirms support.
Audio	Copy. Keep audio behavior simple and fast by default.
Container	Source. Preserve source container choice where supported.
Seek	Accurate. Prefer correctness at chapter boundaries.
Overwrite	Overwrite by default, but emit a human-readable message for each overwritten output path.
Manifest	Write JSON manifest by default. CSV remains optional through config or --csv-manifest.
Verify durations	Keep duration verification on by default unless an explicit --no-verify flag is later approved.
Open output directory	Never open folders from CLI. That GUI behavior should not apply to automation.

```
# First-wave flags
--dry-run
--threads 4
--crf 20
--cq 24
--x264-preset medium
--nvenc-preset p5
--csv-manifest
--stop-on-first-error
--use-gui-config
--embedded
```

Advanced setting restraint

Do not expose encoder/audio/container/seek switches in the first CLI release. Their defaults should be auto, copy, source, and accurate. This preserves behavior while keeping the initial CLI easy to validate.

8. Qt-Free Architecture Boundary

The implementation should introduce a reusable, non-Qt service layer rather than calling Qt services from the CLI. This keeps the CLI shippable without Qt and creates a better long-term foundation for the GUI too.

Layer	Suggested location	Responsibility
Core domain	src/core	Existing models, timecode, chapter validation, default chapters, output planning, command building.
CLI config	src/cli or src/core/config_io	Parse JSON/YAML/CSV/manual chapter inputs into project-owned config aggregates.
Process runner	src/platform or src/core/process	Run ffprobe and ffmpeg without Qt. On Windows, wrap CreateProcessW with pipes.
Probe service	src/services or src/core/probe	Call ffprobe, parse JSON, produce VideoMetadata without QString/QJson/QProcess.
Export engine	src/services or src/core/export	Build jobs, create output directories, run ffmpeg sequentially, parse progress, verify durations, write manifests.
CLI shell	src/cli	Parse argv or shell commands, resolve jobs, print progress, return exit codes.
Qt adapter	src/qt	Can later delegate to the same engine while preserving GUI signals and widgets.

8.1 Dependency Direction

```
src/cli    -> src/services -> src/core
src/qt     -> src/services -> src/core
src/services -> src/core + platform process runner
src/core   -> standard C++ only, no Qt
```

The CLI target should build with VIDCHOPPER_BUILD_GUI=OFF. That is the strongest practical gate that the CLI does not depend on Qt.

9. Process, Probe, and Export Engine

Component	Implementation standard
ProcessRunner	Provide a small interface that starts a process, streams stdout/stderr, returns exit code/status, and supports

Component	Implementation standard
	cancellation later. Windows v1 can use CreateProcessW with redirected pipes.
FfprobeService replacement	Invoke ffprobe with -v error -print_format json -show_format -show_streams -show_chapters, then parse into VideoMetadata using standard C++ and JSON library types.
EncoderEnvironment	Port GPU/encoder detection to Qt-free code. If detection is uncertain, choose the safe x264 fallback through existing resolve_encoder behavior.
ExportEngine	Resolve all jobs first, validate every job, create directories, then export sequentially. Do not partially start a batch before basic job mapping is validated.
Progress parsing	Continue using ffmpeg -progress pipe:1 -nostats and parse out_time_ms for overall progress. Print chapter-level human messages.
Manifest writer	Write vidchopper-manifest.json by default and optional CSV with standard streams. Keep fields aligned with the GUI manifest.

Dependency recommendation

Use nlohmann/json for JSON parsing and yaml-cpp for YAML parsing unless the project later standardizes on another dependency route. YAML support is a product requirement, so hand-rolling YAML is not appropriate.

10. Error Handling, Exit Codes, and Human Output

Situation	Expected behavior
Missing config on normal chop	Fail before export. If embedded chapters exist, print a hint: rerun with --embedded.
Invalid config schema	Fail with filename, line/field when available, and a concise list of invalid fields.
N:N mismatch	Fail before export and list missing config stems, orphan config stems, and duplicate stems.
Chapter validation failure	Fail the affected job with chapter index/name and validation message. Continue other jobs by default unless --stop-on-first-error is set.
ffprobe failure	Fail the job and continue by default in batch mode. Return non-zero overall exit status.
ffmpeg failure	Report source file, chapter index/name, output path, and exit code. Continue by default unless --stop-on-first-error is set.
Overwrite event	Print: Overwriting existing file: <path>. This is informational, not a warning.

Exit code	Meaning
0	All requested jobs completed successfully.
1	Validation or argument error prevented execution.
2	One or more export jobs failed.
3	Tooling error: ffmpeg/ffprobe missing, failed to start, or returned unusable probe data.
4	User cancellation, reserved for future cancellation support.

```

Resolved 3 export job(s).
[1/3] Probing: Sol_Set_01.mkv
[1/3] Exporting 01 - Intro.mp4
Overwriting existing file: D:\out\01 - Intro.mp4
[1/3] Exporting 02 - Match 1.mp4
[1/3] Complete: 2 clips written, manifest written.
[2/3] Skipped: Ky_Set_02.mkv has no embedded chapters. Use a config file or remove --embedded.
Batch finished with warnings: 2 succeeded, 1 skipped, 0 failed.

```

11. Test and Quality Strategy

Testing should prove that the CLI is deterministic before it proves that ffmpeg is fast. Keep most tests pure and reserve slow tests for actual media export.

Test group	Coverage
Fast parser tests	JSON/YAML schema, numeric-ms and string time parsing, missing keys, duplicate chapters, unknown fields policy.
Fast matcher tests	1:1, N:1, N:N, rejected 1:N, duplicate video stems, duplicate config stems, missing pairs, case-insensitive Windows matching.
Fast command tests	Dry-run output, ffmpeg command token order, overwrite flag behavior, output path/naming parity with GUI.
Fast no-Qt build test	CMake config with GUI disabled and CLI enabled must build without finding Qt.
Slow ffmpeg tests	Synthesize sample videos, export chapter clips, verify durations, write manifests.
Embedded chapter tests	Create or fixture a video with embedded chapters, require explicit --embedded, and verify hint behavior when omitted.
Shell tests	Command-loop parsing for chop, help, and exit without needing actual ffmpeg for parser-level tests.

11.1 Quality Gates

- clang-format and clang-tidy rules remain aligned with CODING_STYLE.md.
- Existing fast and slow tests still pass.
- New CLI tests are labeled so pure parser/matcher checks stay fast.
- No Qt headers appear in CLI or shared non-Qt service files.
- Dry-run output has golden coverage to prevent accidental drift.
- Every user-facing error added for CLI has at least one test.

12. Implementation Sequencing

Step	Work item	Acceptance test
1	Add the decision document and update knowledge handoff notes.	Future agents can find the CLI decision, constraints, and examples before coding.
2	Add CMake option and empty VidChopperCLI.exe target.	Build succeeds with GUI disabled; --help and --version work.
3	Introduce Qt-free process runner.	Unit tests cover process success, failure, stdout/stderr capture, and missing executable behavior.
4	Port ffmpeg probing to Qt-free service.	Probe tests produce VideoMetadata equivalent to GUI expectations for duration, frame rate, source extension, and embedded chapters.
5	Add JSON/YAML config model and parser.	Schema tests pass for valid configs and detailed invalid-config failures.
6	Implement job resolver for 1:1, N:1, N:N, and embedded modes.	Matcher tests prove allowed and rejected workload matrices before ffmpeg runs.
7	Implement dry-run.	Dry-run prints planned clips, output paths, and commands without creating outputs.
8	Implement sequential export engine and human progress.	Slow ffmpeg test exports clips and verifies manifests/durations.
9	Add shell mode.	Parser-level shell tests validate chop/help/exit and reuse direct command handling.
10	Integrate Windows packaging.	Release zip contains VidChopper.exe and VidChopperCLI.exe; README documents both.

Pull request policy

Keep each PR narrow: target skeleton, process runner, probing, config parsing, job resolution, dry-run, export, shell mode, packaging, docs. Do not combine the Qt-free refactor with unrelated GUI redesign or Pages changes.

13. Packaging and Documentation

Artifact or doc	Required update
Windows release zip	Include VidChopperCLI.exe beside VidChopper.exe. Do not require Qt runtime files for the CLI itself, even though the zip may include them for the GUI.
README.md	Add CLI quick start, direct mode examples, config schema link, embedded chapter hint, and batch rules.
knowledge/architecture/repo-map.md	Add CLI and non-Qt service layer to the architecture map.
knowledge/history/features-and-progress.md	Track CLI feature status as a new milestone rather than mixing it with the v0.2.0-alpha desktop round.
packaging/windows README	Document that ffmpeg and ffprobe remain external unless a future release chooses to bundle them.
Developer docs site	Add a CLI reference page after the command surface stabilizes.

14. Definition of Done

- VidChopperCLI.exe builds on Windows without Qt when VIDCHOPPER_BUILD_GUI=OFF.
- Direct mode supports 1:1, N:1, N:N, and explicit embedded-chapter mode.
- Single-video plus config-directory 1:N mode is rejected with a clear error.
- Directory-to-directory matching rejects duplicate stems, missing pairs, and orphan configs before exporting.
- JSON and YAML configs normalize into the same internal model and produce equivalent behavior.
- Normal chop without explicit chapters fails and prints an embedded-chapter hint when applicable.
- Dry-run prints planned jobs and ffmpeg commands without exporting files.
- Overwrite is the default and every overwritten file is reported.
- JSON manifest is written by default; CSV is optional.
- Batch mode continues after job failures by default and supports --stop-on-first-error.
- The Windows release zip contains both VidChopper.exe and VidChopperCLI.exe.
- README and knowledge docs describe the CLI workflow, config schema, matching rules, and packaging behavior.

15. Instructions for AI Agents and Contributors

- Do not import Qt from new CLI files or shared non-Qt service files.
- Do not duplicate command-building behavior. Use the existing core command_builder path.
- Add tests before changing matching rules, config schema, overwrite semantics, or ffmpeg argument order.

- Keep user-facing CLI errors deterministic and searchable. Include source/config filenames in batch failures.
- Do not introduce extra ffmpeg arguments, encoder switches, or container switches in the first implementation unless this document is revised.
- Do not silently use embedded chapters. Use --embedded or fail with the hint command.
- When adding dependencies, document why the dependency is needed and how it is packaged on Windows.
- Update knowledge/history when CLI task status changes.

Standing agent note:

- CLI mode is Qt-free.
- Direct command behavior is the public contract.
- Resolve every batch into a validated job list before starting ffmpeg.
- Preserve GUI-aligned naming and command semantics unless the CLI plan explicitly says otherwise.
- Run fast tests before slow ffmpeg tests; do not mark complete with only manual smoke testing.

16. Appendix: Example Configs and Invocations

16.1 Minimal JSON Config

```
{
  "version": 1,
  "chapters": [
    { "name": "Intro", "start": "00:00:00.000", "end": "00:02:30.000" },
    { "name": "Main Set", "start": "00:02:30.000", "end": "00:45:00.000" }
  ]
}
```

16.2 Agent-Friendly Numeric Millisecond Config

```
{
  "version": 1,
  "chapters": [
    { "name": "Intro", "startMs": 0, "endMs": 150000 },
    { "name": "Main Set", "startMs": 150000, "endMs": 2700000 }
  ],
  "settings": {
    "crf": 18,
    "threads": 4
  }
}
```

16.3 Directory N:N Example

```
D:\batch\videos\  
sol_set_01.mkv  
ky_set_02.mp4  
  
D:\batch\chapters\  
sol_set_01.yaml  
ky_set_02.json  
  
VidChopperCLI.exe D:\batch\videos D:\batch\chapters --dry-run  
VidChopperCLI.exe D:\batch\videos D:\batch\chapters
```

16.4 Rejected Duplicate-Stem Example

```
D:\batch\videos\  
evo_pool.mp4  
evo_pool.mkv  
  
Rejected:  
Duplicate video stem "evo_pool" found:  
- D:\batch\videos\evo_pool.mp4  
- D:\batch\videos\evo_pool.mkv  
Choose one source file or rename one file before running directory mode.
```

17. Source Basis and Codebase Notes

Source	How it informed this plan
README.md	Confirmed VidChopper is Windows-first, Qt Widgets based, and built around a Qt-free core plus ffmpeg chapter export.
CMakeLists.txt	Confirmed current target split: vidchopper_core plus optional Qt GUI target. This directly motivates adding a separate CLI executable target.
src/core/models.h	Confirmed ExportSettings, EncoderEnvironment, chapter models, and default settings that the CLI should reuse.
src/core/command_builder.*	Confirmed deterministic ffmpeg command construction that dry-run and export should use.
src/qt/services/*	Confirmed useful service behavior currently tied to Qt process and JSON APIs; this should be refactored into non-Qt services for CLI.
knowledge docs	Confirmed stable contracts include enum values, INI semantics, output filename patterns, and ffmpeg argument ordering.
Attached pdf-it manager guides	Inspired the structure of this document: executive decision, manager directive, implementation sequencing, definition of done, and agent instructions.